CS458 Software Verification and Validation — Project 2 Report

ARES-X: Integrated Adaptive Survey Ecosystem with TDD & Cross-Platform Orchestration

Maksat Abrayev – 22201182 Hüseyin Utku Yüksel – 22103511 Farabi Sina Sarı – 22102084 Furkan Özek – 22103680

Table of Contents

1. Introduction
2. System Overview
    - 2.1 Technology Stack
    - 2.2 Project Structure (Monorepo)
3. Web Architect (TDD)
    - 3.1 TDD Workflow
    - 3.2 Survey Designer Features
    - 3.3 API Integration
4. Mobile Client (Native)
    - 4.1 Login Screen (Project 1 Adaptation)
    - 4.2 Survey Rendering
    - 4.3 Live Schema Polling
5. Algorithmic Orchestration
    - 5.1 DAG Representation
    - 5.2 RCLR – Recursive Conditional Logic Resolution
    - 5.3 GBCR – Graph-Based Conflict Resolution
    - 5.4 Schema Versioning System
6. Verification & Validation Strategy
    - 6.1 TDD Unit Tests (Red-Green-Refactor)
    - 6.2 Appium Mobile Test Cases (10 scenarios)
    - 6.3 Sync-Conflict Automated Test
7. UML Diagrams
    - 7.1 Class Diagram
    - 7.2 Use-Case Diagram
    - 7.3 Sequence Diagram
    - 7.4 State Diagram

    - 7.5 Activity Diagram
8. LLM Integration & Prompts
9. Registered Test Users
10. Environment Variables
11. Limitations

---

1. Introduction

ARES-X (AI-Driven Resilient & Evolutionary Authentication System – eXtended) is an integrated adaptive survey ecosystem that extends our Project 1 authentication platform into a full cross-platform verification environment.

The system comprises:

- **Web Architect**: A responsive survey designer built with strict TDD, allowing creation of multiple question types (single-choice, multi-choice, rating, open-ended text) connected via a DAG structure.
- **Mobile Client**: A native Android application (React Native/Expo) that adapts the Project 1 ARES login page and dynamically renders surveys designed on the web.
- **GBCR/RCLR Algorithms**: Graph-Based Conflict Resolution and Recursive Conditional Logic Resolution algorithms that manage conditional visibility, schema versioning, and mid-session conflict resolution.
- **Automated Testing**: TDD unit tests, 10 Appium mobile test cases, and a synchronized Selenium + Appium cross-platform conflict test.

---

2. System Overview

2.1 Technology Stack

Layer	Technology
Shared Logic	TypeScript — `@ares/survey-core` package
API Server	Fastify 5.x + CORS (Node.js)
Web Architect	Vite + React 19 + TypeScript
Mobile Client	React Native 0.76 + Expo SDK 52
Unit Tests	Vitest (survey-core + web-architect)
Mobile E2E	Appium 2.x via WebdriverIO

Sync Test	Selenium WebDriver (Chrome) + Appium
Build System	npm Workspaces (monorepo)

2.2 Project Structure (Monorepo)

```
CS458-Project2/
├── packages/
│   └── survey-core/          # Shared logic library
│       ├── src/
│       │   ├── types.ts      # SurveySchema, SessionState, RCLR/GBCR types
│       │   ├── dag.ts        # DAG validation (assertAcyclic, edgeSatisfied)
│       │   ├── rclr.ts       # Recursive Conditional Logic Resolution
│       │   └── gbcr.ts       # Graph-Based Conflict Resolution
│       └── tests/
│           ├── dag.test.ts   # 4 tests
│           ├── rclr.test.ts  # 5 tests
│           └── gbcr.test.ts  # 4 tests
├── apps/
│   ├── api/                  # Fastify REST API
│   │   └── src/
│   │       ├── index.ts      # Routes: surveys CRUD, /sync, /debug
│   │       └── store.ts      # In-memory versioned store
│   ├── web-architect/        # Vite + React survey designer
│   │   └── src/
│   │       ├── App.tsx       # Main survey designer UI
│   │       ├── api.ts        # fetchLatestSurvey, publishSurvey
│   │       └── logic/
│   │           ├── architectReducer.ts      # useReducer state management
│   │           ├── architectReducer.test.ts  # 3 TDD tests
│   │           └── architectTypes.ts        # Draft + Action types
│   └── mobile/               # React Native / Expo
│       ├── App.tsx           # Login screen + Survey flow
│       └── index.js          # Entry point
├── e2e/
│   ├── mobile-appium/        # 10 Appium test cases
│   │   └── tests/mobile-logic.test.js
│   └── sync-suite/           # Cross-platform sync conflict test
│       └── sync-conflict.mjs
└── package.json              # Workspace root
```

3. Web Architect (TDD)

3.1 TDD Workflow (Red-Green-Refactor)

The Web Architect was built following strict Test-Driven Development:

Red Phase — Tests were written first, before any functional code:

```
// architectReducer.test.ts — written FIRST (Red)
describe("architectReducer (TDD)", () => {
  it("upserts a question", () => {
    const d = draft();
    const next = architectReducer(d, {
      type: "upsert_question",
      question: { id: "q2", kind: "text", title: "Open" },
    });
    expect(next.schema.questions.q2?.title).toBe("Open");
  });

  it("removes question and incident edges", () => {
    let d = draft();
    d = architectReducer(d, {
      type: "upsert_question",
      question: { id: "q2", kind: "text", title: "X" },
    });
    d = architectReducer(d, {
      type: "add_edge",
      edge: { id: "e1", from: "q1", to: "q2" },
    });
    const next = architectReducer(d, { type: "remove_question", id: "q2" });
    expect(next.schema.edges).toHaveLength(0);
    expect(next.schema.questions.q2).toBeUndefined();
  });

  it("loads schema from API", () => {
    const loaded = baseSchema();
    loaded.version = 4;
    const next = architectReducer(draft(), {
      type: "load_schema", schema: loaded
    });
    expect(next.schema.version).toBe(4);
```

```
  });
});
```

Green Phase — Minimal implementation to make tests pass:

```
// architectReducer.ts — implemented SECOND (Green)
export function architectReducer(
  state: ArchitectDraft,
  action: ArchitectAction
): ArchitectDraft {
  switch (action.type) {
    case "upsert_question": {
      const questions = {
        ...state.schema.questions,
        [action.question.id]: action.question
      };
      return { ...state, schema: { ...state.schema, questions } };
    }
    case "remove_question": {
      const { [action.id]: _, ...rest } = state.schema.questions;
      const edges = state.schema.edges.filter(
        (e) => e.from !== action.id && e.to !== action.id,
      );
      const entryId = state.schema.entryId === action.id
        ? Object.keys(rest)[0] ?? ""
        : state.schema.entryId;
      return {
        ...state,
        selectedQuestionId:
          state.selectedQuestionId === action.id
            ? undefined : state.selectedQuestionId,
        schema: { ...state.schema, questions: rest, edges, entryId },
      };
    }
    case "load_schema":
      return { ...state, schema: action.schema, selectedQuestionId: undefined
};
    // ... other cases
```

```
  }
}
```

Refactor Phase — Actions were extended with `add_edge`, `remove_edge`, `set_entry`, and `select_question` while maintaining all tests green.

3.2 Survey Designer Features

The Web Architect provides:

- **Add Question**: Creates new questions with unique IDs (collision-safe generation)
- **Remove Question**: Removes a question and all incident edges (✕ button per card)
- **Edit Question**: Inline editing of title, kind (single_choice, multi_choice, rating, text), and options
- **Link Edges**: "Link last two" button creates conditional edges between questions
- **Remove Edge**: Per-edge remove button
- **DAG Validation**: `assertAcyclic()` is called before every publish to prevent cycles
- **Publish**: Sends the schema to the API, which bumps the version number

3.3 API Integration

The Fastify API exposes:

Method	Endpoint	Purpose
GET	`/surveys/:id`	Fetch latest schema version
GET	`/surveys/:id/versions/:v`	Fetch specific version
PUT	`/surveys/:id`	Publish new version (validates DAG)
POST	`/surveys/:id/sync`	GBCR sync for mobile client
GET	`/health`	Health check

The `store.ts` maintains an in-memory history of all published versions, enabling the GBCR algorithm to compare any two schema versions during migration.

---

4. Mobile Client (Native)

4.1 Login Screen (Project 1 Adaptation)

The mobile login screen directly adapts the Project 1 ARES authentication page:

Project 1 (Web) Feature	Mobile Adaptation
ARES branding with logo icon	`A` badge + "ARES-X" text
Email/Phone tab toggle	Email/Phone tab buttons with active highlighting
Email/password input fields	`TextInput` with keyboard types
Google OAuth button	"Login with Google" `TouchableOpacity`
GitHub OAuth button	"Login with GitHub" `TouchableOpacity`
Dark gradient theme	Dark background (`#0f172a`) with indigo accents
Test account info	Shown at bottom of login screen

Key login screen code:

```
function LoginScreen({ onLogin }: { onLogin: () => void }) {
  const [loginType, setLoginType] = useState<"email" | "phone">("email");
  const [identifier, setIdentifier] = useState("");
  const [pass, setPass] = useState("");

  return (
    <ScrollView testID="login-screen">
      {/* ARES-X Branding */}
      <View style={loginStyles.logoRow}>
        <View style={loginStyles.logoIcon}>
          <Text style={loginStyles.logoLetter}>A</Text>
        </View>
        <Text style={loginStyles.logoText}>ARES-X</Text>
      </View>

      {/* Email / Phone tabs (adapted from Project 1) */}
      <View style={loginStyles.tabRow} testID="login-tabs">
        <TouchableOpacity
          style={[loginStyles.tab,
            loginType === "email" && loginStyles.tabActive]}
          onPress={() => { setLoginType("email"); }}
          testID="tab-email"
        >
          <Text>📧 Email</Text>
        </TouchableOpacity>
```

```
        <TouchableOpacity
          style={[loginStyles.tab,
            loginType === "phone" && loginStyles.tabActive]}
          onPress={() => { setLoginType("phone"); }}
          testID="tab-phone"
        >
          <Text>📱 Phone</Text>
        </TouchableOpacity>
      </View>

      {/* Credential inputs + Social auth buttons */}
      <TextInput testID="login-user" ... />
      <TextInput testID="login-pass" secureTextEntry ... />
      <TouchableOpacity testID="login-submit" ... />
      <TouchableOpacity testID="google-login-btn" ... />
      <TouchableOpacity testID="github-login-btn" ... />
    </ScrollView>
  );
}
```

4.2 Survey Rendering

After login, the mobile app renders the survey dynamically:

- **single_choice**: Buttons for each option (highlighted when selected)
- **multi_choice**: Checkbox-style toggles with visual checkmark
- **text**: Free-form text input
- **rating**: Numeric buttons (1–N, highlighted when selected)
- **Send button**: Only appears when all required visible questions are answered (gated by RCLR `canShowSubmit`)

4.3 Live Schema Polling

The mobile client polls the API every **2.5 seconds** for schema updates:

```
useEffect(() => {
  const id = setInterval(async () => {
    const latest = await fetchLatest();
    if (latest.version === cur.version) return;
    // GBCR migration
    const migrated = migrateSessionForNewSchema(cur, latest, sess);
```

```
    setGbcr(migrated);
    setSchema(latest);
    setSession(migrated.session);
    // RCLR check
    const rclr = resolveVisibility(latest, migrated.session.answers);
    setRclrNote(rclr.consistent ? null : rclr.conflictCodes.join(", "));
  }, 2500);
  return () => clearInterval(id);
}, []);
```

When a schema change is detected, the GBCR banner shows the migration outcome (atomic recovery or rollback) and RCLR conflict codes — this is **not a simple popup** but a structured conflict display.

---

5. Algorithmic Orchestration

5.1 DAG Representation

The survey is modeled as a **Directed Acyclic Graph (DAG)** where:

- **Nodes** = Questions (each with an `id`, `kind`, `title`, `options`, `required`)
- **Edges** = Conditional transitions (`from → to` with optional condition)
- **Entry** = The root node (`entryId`) where the survey starts

```
export type SurveySchema = {
  id: string;
  version: number;
  entryId: string;
  questions: Record<string, SurveyQuestion>;
  edges: SurveyEdge[];
};

export type SurveyEdge = {
  id: string;
  from: string;
  to: string;
  condition?: EdgeCondition;  // equals | oneOf
};
```

Cycle detection prevents invalid graphs at publish time:

```
export function assertAcyclic(schema: SurveySchema): void {
  const graph = adjacencyOutgoing(schema.edges);
  const visiting = new Set<string>();
  const visited = new Set<string>();

  const dfs = (n: string, stack: Set<string>) => {
    if (visited.has(n)) return;
    if (stack.has(n)) {
      throw new Error(`CYCLE_DETECTED at ${n}`);
    }
    stack.add(n);
    for (const next of graph.get(n) ?? []) {
      dfs(next, stack);
    }
    stack.delete(n);
    visited.add(n);
  };

  dfs(schema.entryId, visiting);
  for (const q of Object.keys(schema.questions)) {
    if (!visited.has(q)) dfs(q, visiting);
  }
}
```

5.2 RCLR – Recursive Conditional Logic Resolution

RCLR computes which questions are **visible** given the current answers by traversing the DAG from the entry node:

```
export function resolveVisibility(
  schema: SurveySchema,
  answers: SessionAnswers,
): RclrResult {
  const visible = new Set<string>();
  const conflictCodes: RclrConflictCode[] = [];

  const visit = (nodeId: string) => {
    visible.add(nodeId);
    const outs = edgesByFrom.get(nodeId) ?? [];
```

```
    for (const e of outs) {
      if (!edgeSatisfied(e.condition, answers)) continue;
      visit(e.to);
    }
  };

  visit(schema.entryId);

  // Detect orphans, stale answers, and unknown question refs
  if (!orphanHeuristic(schema, answers, visible))
    conflictCodes.push("ORPHAN_VISIBLE");

  for (const qid of Object.keys(answers)) {
    if (schema.questions[qid] && !visible.has(qid))
      conflictCodes.push("STALE_ANSWER");
  }

  return {
    visibleIds: visible,
    consistent: conflictCodes.length === 0,
    conflictCodes
  };
}
```

Conflict codes detected by RCLR:

Code	Meaning
`ORPHAN_VISIBLE`	A visible node has no valid incoming path from entry
`MISSING_ENTRY`	The schema's entry node doesn't exist in questions
`CYCLE_DETECTED`	The DAG contains a cycle
`CONDITION_UNKNOWN_QUESTION`	An edge condition references a non-existent question
`STALE_ANSWER`	An answer exists for a question that is not currently visible

Send button gate: The `canShowSubmit()` function returns `true` only when RCLR is consistent AND all required visible questions have been answered.

5.3 GBCR – Graph-Based Conflict Resolution

When the mobile client detects a schema version change during a live session, GBCR determines how to migrate:

Outcome A: Atomic State Recovery (`recovered_atomic`)

- Maps existing answers to the new DAG structure
- Drops answers for deleted questions
- Verifies consistency with RCLR
- No data loss for valid answers

Outcome B: Rollback to Stable Node (`rollback_stable`)

- RCLR detects inconsistency in the migrated state
- Walks the session trail backwards to find the last node where RCLR was consistent
- Trims answers and trail to that stable point
- Reports conflict codes to the UI

```
export function migrateSessionForNewSchema(
  prevSchema: SurveySchema,
  nextSchema: SurveySchema,
  session: SessionState,
): GbcrMigrationResult {
  // 1. If version matches, no migration needed
  if (session.schemaVersion === nextSchema.version)
    return { outcome: "unchanged", session };

  // 2. Drop answers for deleted questions
  const nextAnswers = { ...session.answers };
  for (const qid of Object.keys(nextAnswers)) {
    if (!nextSchema.questions[qid]) {
      delete nextAnswers[qid];
      dropped.push(qid);
    }
  }

  // 3. Check consistency with RCLR
  const rclr = resolveVisibility(nextSchema, nextAnswers);
  if (rclr.consistent) {
    return { outcome: "recovered_atomic", session: provisional, ... };
  }

  // 4. Rollback: find last stable node
```

```
  const stable = findLastStableNode(nextSchema, nextAnswers, trail);
  return { outcome: "rollback_stable", stableNodeId: stable, ... };
}
```

5.4 Schema Versioning System

The API server maintains a complete version history:

```
// store.ts
export function publish(schema: SurveySchema): SurveySchema {
  const cur = surveys.get(schema.id);
  const nextVersion = cur
    ? Math.max(...cur.history.map((h) => h.version)) + 1
    : schema.version;
  const next = { ...schema, version: nextVersion };
  const history = cur ? [...cur.history, next] : [next];
  surveys.set(schema.id, { history });
  return next;
}
```

The `/surveys/:id/sync` endpoint performs GBCR migration server-side, comparing the client's reported schema version against the latest published version.

---

6. Verification & Validation Strategy

6.1 TDD Unit Tests (Red-Green-Refactor)

16 unit tests across 4 test files — all passing:

Test File	Tests	Layer
`dag.test.ts`	4	Cycle detection, acyclic validation, edge satisfaction, adjacency
`rclr.test.ts`	5	Visibility resolution, branch expansion, cycle flags, orphan detection, submit gate
`gbcr.test.ts`	4	Version match, atomic recovery, rollback, empty trail edge case

`architectReducer.test.ts`	3	Upsert question, remove question + edges, load schema

DAG tests excerpt:

```
describe("dag helpers", () => {
  it("detects cycles", () => {
    const edges = [
      { id: "e1", from: "q1", to: "q2" },
      { id: "e2", from: "q2", to: "q3" },
      { id: "e3", from: "q3", to: "q1" },  // creates cycle
    ];
    expect(() => assertAcyclic(baseSchema(edges)))
      .toThrow("CYCLE_DETECTED");
  });

  it("edgeSatisfied respects equals and oneOf", () => {
    const answers = { q1: "yes" };
    expect(edgeSatisfied(
      { type: "equals", questionId: "q1", value: "yes" }, answers
    )).toBe(true);
    expect(edgeSatisfied(
      { type: "oneOf", questionId: "q1", values: ["no", "maybe"] }, answers
    )).toBe(false);
  });
});
```

RCLR tests excerpt:

```
describe("RCLR visibility", () => {
  it("recursively expands branch A", () => {
    const schema = mk([
      { id: "e1", from: "q1", to: "q2",
        condition: { type: "equals", questionId: "q1", value: "a" } },
      { id: "e2", from: "q1", to: "q3",
        condition: { type: "equals", questionId: "q1", value: "b" } },
    ]);
    const r = resolveVisibility(schema, { q1: "a" });
    expect(r.visibleIds.has("q1") && r.visibleIds.has("q2")).toBe(true);
    expect(r.visibleIds.has("q3")).toBe(false);
```

```
    expect(r.consistent).toBe(true);
  });

  it("submit hidden until required visible questions answered", () => {
    const r1 = resolveVisibility(schema, { q1: "a" });
    expect(canShowSubmit(schema, { q1: "a" }, r1)).toBe(false);
    expect(canShowSubmit(schema, { q1: "a", q2: "ok" }, r1)).toBe(true);
  });
});
```

GBCR tests excerpt:

```
describe("GBCR schema versioning", () => {
  it("atomic recovery drops answers for deleted nodes", () => {
    const prev = schemaV1();
    const next = { ...prev, version: 2,
      questions: { q1: prev.questions.q1, q3: prev.questions.q3 },
      edges: [/* updated edges */]
    };
    const sess = sessionMid(); // answers: { q1: "x", q2: "hello" }
    const r = migrateSessionForNewSchema(prev, next, sess);
    expect(r.outcome).toBe("recovered_atomic");
    expect(r.droppedAnswersFor).toContain("q2");
    expect(r.session.answers.q1).toBe("x");
    expect(r.session.answers.q2).toBeUndefined();
  });

  it("rollback when logic becomes inconsistent", () => {
    // Swap edge conditions so q2 requires "y" instead of "x"
    const next = { ...prev, version: 3, edges: [/* swapped */] };
    const r = migrateSessionForNewSchema(prev, next, sess);
    expect(r.outcome).toBe("rollback_stable");
    expect(r.conflictCodes.length).toBeGreaterThan(0);
  });
});
```

6.2 Appium Mobile Test Cases (10 Scenarios)

All 10 test cases focus on **non-trivial logic** rather than simple UI interactions:

#	Test Case	What It Verifies
01	Login screen exposes username field	Login UI accessible via testID
02	Successful login reveals survey shell	Login → Survey transition
03	Schema version label tracks backend	Real-time API version tracking
04	First question renders choices	DAG entry node rendering
05	Choosing path A reveals Path A detail	**Recursive visibility** (RCLR)
06	Path B rating branch hidden when A selected	**Mutual exclusion** in DAG branches
07	Text completion enables Send (RCLR consistent)	**canShowSubmit gate**
08	GBCR banner uses structured outcome codes	**GBCR migration** UI after hot publish
09	RCLR conflict strip not a generic alert	Structured conflict display (not popup)
10	No zombie question without parent	**ORPHAN_VISIBLE** detection

Test excerpt — Recursive visibility (Test 05):

```
req("05 choosing path A reveals Path A detail (recursive visibility)",
  async () => {
    await driver.$('~choice-q1-A').click();
    const q2 = await driver.$('~question-q2');
    assert.ok(await q2.isDisplayed());
  }
);
```

Test excerpt — No zombie question (Test 10):

```
req("10 no zombie question without parent (visibility tied to DAG)",
  async () => {
    const orphan = await driver.$('~question-orphan');
    const exists = await orphan.isDisplayed().catch(() => false);
    assert.equal(exists, false);
  }
);
```

6.3 Sync-Conflict Automated Test (Selenium + Appium)

The sync-conflict test verifies cross-platform consistency:

Scenario: While the Appium script selects an answer on mobile, the Selenium script simultaneously publishes a conflicting schema change on the Web Architect. The test verifies:

1. The mobile app does **not** enter an undefined UI state
2. The GBCR banner or RCLR conflict strip appears (structured, not a generic popup)
3. Mutually exclusive branches do not both appear visible

```
// sync-conflict.mjs
const conflicting = {
  ...before,
  edges: [
    // SWAP: A now goes to q3, B now goes to q2 (opposite)
    { id: "e1", from: "q1", to: "q3",
      condition: { type: "equals", questionId: "q1", value: "A" } },
    { id: "e2", from: "q1", to: "q2",
      condition: { type: "equals", questionId: "q1", value: "B" } },
  ],
};

await Promise.all([
  // Web leg: publish conflicting schema
  (async () => {
    await new Promise((r) => setTimeout(r, 3500)); // delay
    await runWebLeg(conflicting);
  })(),
  // Mobile leg: select answer + verify conflict handling
  APP ? runMobileLeg() : Promise.resolve(),
]);
```

Mobile verification in sync test:

```
const bannerOn = await banner.isDisplayed().catch(() => false);
const conflictOn = await conflict.isDisplayed().catch(() => false);
assert.ok(
  bannerOn || conflictOn,
  "Expected GBCR banner or RCLR conflict strip"
```

```
);

// Verify no zombie state
const both = (await q2.isDisplayed().catch(() => false))
  && (await q3.isDisplayed().catch(() => false));
assert.equal(both, false,
  "Mutually exclusive branches should not both appear");
```

---

7. UML Diagrams

7.1 Class Diagram

```
┌──────────────────────────────┐
│        SurveySchema          │
├──────────────────────────────┤
│ + id: string                 │
│ + version: number            │
│ + entryId: string            │
│ + questions: Record<id, Q>   │
│ + edges: SurveyEdge[]        │
├──────────────────────────────┤
│ + assertAcyclic(): void      │
└──────────────────────────────┘
           │
           │ 1..*
           ▼

┌──────────────────────────────┐      ┌──────────────────────────────┐
│        SurveyQuestion        │      │        SurveyEdge            │
├──────────────────────────────┤      ├──────────────────────────────┤
│ + id: string                 │      │ + id: string                 │
│ + kind: QuestionKind         │      │ + from: string               │
│ + title: string              │      │ + to: string                 │
│ + options?: string[]         │      │ + condition?: EdgeCond.      │
│ + maxRating?: number         │      ├──────────────────────────────┤
│ + required?: boolean         │      │ + edgeSatisfied(): bool      │
└──────────────────────────────┘      └──────────────────────────────┘


┌──────────────────────────────┐      ┌──────────────────────────────┐
│        SessionState          │      │      RclrResult              │
├──────────────────────────────┤      ├──────────────────────────────┤
```

```
│ + schemaId: string          │      │ + visibleIds: Set<string> │
│ + schemaVersion: number     │      │ + consistent: boolean     │
│ + answers: Record<id, val>  │      │ + conflictCodes: Code[]   │
│ + trail: string[]           │      └───────────────────────────┘
│ + currentQuestionId?: str   │
└─────────────────────────────┘
                                     ┌───────────────────────────┐
                                     │   GbcrMigrationResult     │
┌─────────────────────────────┐      ├───────────────────────────┤
│     ArchitectDraft          │      │ + outcome: unchanged |    │
├─────────────────────────────┤      │   recovered_atomic |      │
│ + schema: SurveySchema      │      │   rollback_stable         │
│ + selectedQuestionId?: str  │      │ + session: SessionState   │
├─────────────────────────────┤      │ + droppedAnswersFor?: []  │
│ + architectReducer(action)  │      │ + stableNodeId?: string   │
└─────────────────────────────┘      │ + conflictCodes?: Code[]  │
                                     └───────────────────────────┘
```

7.2 Use-Case Diagram

```
                    ┌──────────────────────────────┐
                    │         ARES-X System        │
                    │                              │
 ┌───────┐          │  ┌────────────────────────┐  │
 │Admin  │──────────┼─▶│ Design survey (Web)    │  │
 │(Web)  │          │  └────────────┬───────────┘  │
 └───────┘          │               │              │
      │             │  ┌────────────▼───────────┐  │
      │─────────────┼─▶│ Add/Edit/Remove questions│  │
      │             │  └────────────┬───────────┘  │
      │             │               │              │
      │─────────────┼─▶┌────────────▼──────────┐   │
      │             │  │ Link edges (conditions) │  │
      │             │  └────────────┬───────────┘  │
      │             │               │              │
      │─────────────┼─▶┌────────────▼──────────┐   │
                    │  │ Publish schema (DAG val)│  │
                    │  └────────────────────────┘  │
                    │                              │
 ┌───────┐          │  ┌────────────────────────┐  │
 │User   │──────────┼─▶│ Login (P1 adapted)     │  │
```

```
│(Mob.)│          │  └──────────┬──────────┘       │
└──────┘          │             │                   │
      │           │  ┌──────────▼──────────┐       │
      │──────────┼─▶│ Fill survey (RCLR vis.) │      │
      │           │  └──────────┬──────────┘       │
      │           │             │                   │
      │──────────┼─▶ ┌──────────▼──────────┐      │
      │           │  │ Submit (canShowSubmit)  │      │
      │           │  └──────────────────────┘       │
      │           │                                 │
      │──────────┼─▶ ┌──────────────────────┐      │
                  │  │ Handle schema conflict  │      │
                  │  │ (GBCR banner/RCLR codes)│      │
                  │  └──────────────────────┘       │
                  └─────────────────────────────────┘
```

7.3 Sequence Diagram — Schema Conflict During Mobile Session

```
Admin(Web)          API Server          Mobile Client
    │                   │                   │
    │                   │    GET /surveys   │
    │                   │◄──────────────────│ (poll every 2.5s)
    │                   │──────────────────▶│ schema v1
    │                   │                   │
    │                   │                   │ User answers q1="A"
    │                   │                   │ RCLR: q1,q2 visible
    │                   │                   │
    │ PUT /surveys/id   │                   │
    │ (swap edges)      │                   │
    │──────────────────▶│ publish v2         │
    │◄──────────────────│                   │
    │                   │                   │
    │                   │    GET /surveys   │
    │                   │◄──────────────────│ (next poll)
    │                   │──────────────────▶│ schema v2
    │                   │                   │
    │                   │                   │ GBCR: migrateSession(v1→v2)
    │                   │                   │ RCLR: STALE_ANSWER detected
    │                   │                   │ outcome: rollback_stable
    │                   │                   │
```

```
    │                    │                    │ UI: GBCR banner + conflict codes
    │                    │                    │ (NOT a generic popup)
```

7.4 State Diagram — Survey Session States

```
┌──────────┐  login success  ┌──────────────┐
│  Login   │──────────────────▶│   Loading    │
└──────────┘                  │   Survey     │
                              └──────┬───────┘
                                     │ schema fetched
                                     ▼
                              ┌──────────────┐
                   ┌──────────▶│  Answering   │◀──────────┐
                   │          │  (RCLR ok)   │           │
                   │          └──────┬───────┘           │
                   │                 │                   │
                   │        ┌────────┴────────┐          │
                   │        ▼                 ▼          │
                   │  ┌──────────────┐  ┌──────────────┐  │
                   │  │ All required │  │ Schema v++   │  │
                   │  │  answered    │  │ detected     │  │
                   │  └──────┬───────┘  └──────┬───────┘  │
                   │         │                 │          │
                   │         ▼                 ▼          │
                   │  ┌──────────────┐  ┌──────────────┐  │
                   │  │  Send Visible│  │  GBCR Run    │  │
                   │  │  (submit OK) │  │              │  │
                   │  └──────┬───────┘  └──────┬───────┘  │
                   │         │                 │          │
                   │         ▼            ┌────┴────┐     │
                   │  ┌──────────────┐    ▼         ▼     │
                   │  │  Submitted   │ recovered  rollback │
                   │  └──────────────┘ _atomic    _stable  │
                   │                      │         │      │
                   │                      └────┬────┘      │
                   │                           └───────────┘
```

7.5 Activity Diagram — Survey Completion Flow

```
┌──────────┐
│   Start  │
└──────────┘
      │
      ▼

┌──────────────────┐
│  Login (Email/   │
│  Phone + Password │
│  or Social Auth) │
└──────────────────┘
         │
         ▼

┌──────────────────┐
│  Fetch Schema    │
│  from API        │
└──────────────────┘
         │
         ▼

┌──────────────────┐◄──────────────────┐
│  Compute RCLR    │                   │
│  (visible nodes) │                   │
└──────────────────┘                   │
         │                             │
         ▼                             │
┌──────────────────┐                   │
│  Render Visible  │                   │
│  Questions       │                   │
└──────────────────┘                   │
         │                             │
         ▼                             │
◆ User answers a question ◆              │
         │                             │
         ├─── Schema v++ detected? ──▶  │
         │    Yes → Run GBCR           │
         │          ├─ atomic → continue ┘
         │          └─ rollback → trim   ┘
         │
         ▼
◆ All required visible answered? ◆
         │
    No ──┘
         │ Yes
         ▼

┌──────────────────┐
│  Show "Send"     │
│  Button          │
```

```
└──────────┬──────────┘
           ▼
┌──────────────────┐
│  Submit Survey   │
└──────────┬──────────┘
           ▼
┌──────────┐
│   End    │
└──────────┘
```

---

8. LLM Integration & Prompts

The following LLM tools were used during development:

LLM	Version	Usage
Google Gemini	Antigravity (Gemini 3 Flash)	Code generation, architecture, testing, report assistance

Key LLM Prompts Used

Prompt 1 — Initial Architecture Design:

> *"Design a monorepo structure for an integrated survey ecosystem with a shared core package, a Fastify API, a Vite React web architect, and an Expo React Native mobile client. The survey should be represented as a DAG with conditional edges."*

Prompt 2 — GBCR Algorithm:

> *"Implement a Graph-Based Conflict Resolution algorithm that handles mid-session schema versioning. When the mobile client is mid-session and the schema changes, the algorithm should either perform atomic state recovery or gracefully rollback to the last stable node."*

Prompt 3 — RCLR Algorithm:

> *"Implement a Recursive Conditional Logic Resolution function that treats the survey as a DAG, recursively calculates question visibility based on current answers, detects orphan nodes, stale answers, cycles, and unknown question references."*

Prompt 4 — TDD Test Cases:

> *"Write vitest test cases following Red-Green-Refactor TDD for the architectReducer. Tests should cover: upsert question, remove question with cascading edge deletion, and loading a schema from the API."*

Prompt 5 — Appium Test Scenarios:

> *"Create 10 comprehensive Appium test cases for the mobile survey client focused on non-trivial logic: recursive visibility, mutual exclusion, GBCR migration banners, RCLR conflict detection (not popup), and zombie question prevention."*

Prompt 6 — Mobile Login Adaptation:

> *"Adapt the Project 1 ARES login page for React Native mobile. Keep the same structure: ARES branding, Email/Phone toggle tabs, social auth buttons (Google, GitHub), dark theme with indigo accents."*

---

9. Registered Test Users

The application uses any credentials for login (demo mode). Test scenarios use:

Username	Password	Purpose
`demo`	`demo`	General testing
`sync`	`sync`	Sync-conflict test
`testuser@ares.com`	`Test@1234`	Project 1 compatibility test
`admin@ares.com`	`Admin@5678`	Admin flow testing

---

10. Environment Variables

Variable	Purpose	Default
`PORT`	API server port	`4000`
`HOST`	API server bind address	`0.0.0.0`
`VITE_API_URL`	Web architect API base URL	`/api` (proxied)
`EXPO_PUBLIC_API_URL`	Mobile API base URL	`http://10.0.2.2:4000`
`APPIUM_APP_PATH`	Path to built APK for Appium	(required for e2e)
`APPIUM_HOST`	Appium server hostname	`127.0.0.1`
`APPIUM_PORT`	Appium server port	`4723`
`WEB_URL`	Web architect URL for sync test	`http://127.0.0.1:5173`

`API_URL`	API URL for sync test	`http://127.0.0.1:4000`

---

11. Limitations

- **No persistent database**: State resets on server restart (in-memory store).
- **Mobile login is demo-only**: Accepts any credentials (no actual authentication backend connected).
- **Appium tests require running emulator and built APK**: Cannot run in pure CI without device farm.
- **Schema polling interval (2.5s)**: May miss very rapid concurrent changes.
- **No real OAuth integration on mobile**: Social auth buttons are visual adaptations of P1; actual OAuth requires native SDK integration.

Report prepared for CS458 Software Verification and Validation — Spring 2025-2026